Claude Code Project Best Practices Guide

A comprehensive guidance document based on the PSB (Plan, Setup, Build) system for starting Claude Code projects effectively.

Phase 1: PLAN (Before Writing Code)

> "Spending just 15 minutes planning your project up front will save you hours, sometimes even days of frustration down the line."

Step 1: Ask Yourself Two Critical Questions

Question 1: What are you actually trying to do?

Your answer fundamentally changes how you approach the project:

Goal	Approach
Learning a new technology	Focus on understanding patterns, skip production polish
Validating an idea with customers	Build just enough to test the hypothesis
Building an alpha version	Include security, error handling, edge cases
Prototyping to see if something works	Move fast, break things, focus on core functionality

Example: If you're prototyping a feature idea, you don't need production-ready code or edge case handling. But if you're shipping to real users, think about security, error handling, and polish.

Question 2: What are the milestones of functionality?

Break your project into clear phases:

- **MVP (v1)**: The absolute minimum to validate your idea or prove the concept works
- **v2**: Scaffolding, polish, and core improvements
- **v3+**: Optimizations, additional features, scaling

Pro tip: Define what "done" looks like for each milestone so you have a clear target.

Step 2: Use AI to Help You Plan

Technique 1: Make AI Ask You Questions

Instead of trying to explain everything upfront, give Claude your rough brainstorm and ask:

> "Hey Claude, I want to build this project. What are the three most important questions I need to answer in order to build an MVP of this successfully?"

This helps you:

- Spot things you haven't fully thought through
- Surface decisions that need to be made
- Get clarity about what you're actually building

You can repeat this for any aspect: product requirements, technical choices, debugging issues.

Technique 2: Voice Conversations

Use Claude's voice mode or ChatGPT voice to talk through fuzzy ideas:

1. Talk out loud about features, user flows, and ideas
2. Ask AI to summarize the conversation in markdown
3. Use that summary as a starting point for Claude Code

This is especially useful when ideas are fuzzy and hard to write down.

Step 3: Create Your Project Spec Document

The project spec has two main parts and serves as your primary planning deliverable.

Part 1: Product Requirements (What & Why)

Answer three key questions:

1. **Who is this product for?**
2. **What problems does it solve?**
3. **What should the product do?**

Be specific about user interactions. If you're vague, Claude will make assumptions you might not like.

❌ Vague	✅ Specific
"Users can create journal entries"	"Users start with a blank page, can add photos, can edit past entries, workflow is: tap → write

	→ save"
"Add authentication"	"Users sign up with email/password, receive verification email, can reset password via email link"

Don't build everything at once. Reference your milestones, what's most important first? What can wait for v2?

Part 2: Technical/Engineering Requirements (How)

Choose Your Tech Stack Explicitly

If you don't specify, Claude might inject random technology choices. Be explicit:

Example Web App Stack:

```
Frontend:
- Hosting: Vercel
- Framework: Next.js
- Styling: Tailwind CSS
- Components: shadcn/ui

Backend:
- Database: MongoDB or Supabase
- Authentication: Clerk
- Payments: Stripe
- Email: Resend
- Object Storage: Cloudflare R2

AI Features:
- LLM: Anthropic Claude
- Images: Google Gemini (Nano Banana)
```

If you're unsure what to use: Ask Claude to create a research report with options and recommendations. It has web search built-in for current docs and frameworks.

Define Your Technical Architecture

Include:

- System design overview
- Key components and how they interact
- Database schema
- API design
- Other technical details that matter

Important: Once you've decided on infrastructure, provision it now:

- Create your database
- Set up hosting

- Generate API keys

This lets Claude move fast when building starts.

Phase 2: SETUP (Configure Claude Code for Success)

Setup Checklist

Set Up Your CLAUDE.md File

Think of this as your project's memory, always included in context for every chat.

Key principle: Keep it focused. Don't bloat it. Link to other docs instead of repeating information.

What to include:

```
# Project Overview
Brief description of goals and current state

# Architecture
Link to architecture.md for full details
Key folders: /src/components, /src/api, /src/lib

# Design Guidelines
- Use shadcn/ui components
- Follow existing color scheme in tailwind.config
- Mobile-first responsive design

# Constraints & Policies
- NEVER push directly to main
- ALWAYS use environment variables for secrets
- Create feature branches for new work

# Repository Etiquette
- Use PRs for features
- Commit messages: conventional commits format
- Branch naming: feature/*, bugfix/*, hotfix/*

# Common Commands
- Build: `npm run build`
- Test: `npm test`
- Dev: `npm run dev`

# Key Files Reference
- Project spec: /docs/project-spec.md
- Architecture: /docs/architecture.md
- Changelog: /docs/changelog.md
```

Pro tip: Link to files rather than copying their content:

> "For full architecture details, see /docs/architecture.md"

These docs are separate from CLAUDE.md and provide deeper context Claude can reference when needed.

Document 1: architecture.md

- System design
- App structure
- How major components interact
- Update after adding big features

Document 2: changelog.md

- List of changes over time
- Helps see what work has been done
- Gives Claude overview of project evolution

Example format:

```
## [2024-01-15]
### Added
- User authentication with Clerk
- Dashboard page with usage metrics

### Changed
- Migrated from localStorage to Supabase

### Fixed
- Mobile navigation menu not closing
```

Document 3: project-status.md

- Project milestones
- What's accomplished for each milestone
- Where you left off last time

This is invaluable for picking up projects after days or weeks away.

Document 4: Feature Reference Docs (Optional)

- High-level overview of complex features
- Examples: onboarding flow, payment system, notification logic
- Helps plan improvements or fix bugs

Keeping docs updated: Create a custom slash command or sub-agent that updates docs after finishing features.

Set Up Plugins

Plugins extend Claude Code with specialized commands and workflows.

Recommended plugins:

Plugin	Purpose
Anthropic Frontend	Better UI generation, avoids generic purple gradients
Anthropic Feature Dev	Streamlined feature development process

Managing plugins: Use `/plugins` command in Claude Code

Set Up MCPs (Model Context Protocol Servers)

MCPs connect Claude Code to external tools and services.

Match MCPs to your tech stack:

Category	MCP Options
Browser Testing	Playwright MCP, Puppeteer MCP
Project Management	Atlassian MCP

Set Up Slash Commands and Sub-Agents

Understanding the difference:

Slash Commands	Sub-Agents
Shortcut to a prompt/task	Specialized agent for specific tasks
Same context window as main conversation	Fork of context window (isolated)
Good for quick actions	Perfect for parallel work and focused tasks

Built-in commands to use:

- `/commit` - Commit changes
- `/pr` - Create pull requests

- Planning and codebase search sub-agents

Recommended custom sub-agents:

1. **Changelog Agent**: Creates/updates changelog entries after features
2. **Frontend Testing Agent**: Runs Playwright tests automatically
3. **Retro Agent**: Reflects on improvements after sessions, updates CLAUDE.md with learnings

Example custom slash command ideas:

- Update all project docs based on recent changes
- Create GitHub issues from a project spec
- Generate a summary of current branch changes

Optional/Advanced Setup Steps

Preconfigure Permissions

Pre-approve or pre-block commands so Claude doesn't wait for permission:

```
{
  "permissions": {
    "allow": ["git *", "npm *", "edit files"],
    "block": ["rm -rf", "push --force"]
  }
}
```

Set Up Hooks

Hooks insert automation at specific points in the Claude Code lifecycle:

Example hooks:

Hook Type	Example
Stop hook	Check if tests pass when Claude finishes; if not, tell Claude to keep going
Notification hook	Ping Slack when Claude needs permission
Pre-commit hook	Run linting before commits

Phase 3: BUILD (Development Workflows)

Building Your MVP

1. Point Claude to your project spec document

2. **Always use plan mode first** for better results
3. Ask Claude to use parallel sub-agents where possible

Example prompt:

> "Based on my project spec in /docs/project-spec.md, build milestone 1 (MVP). Use plan mode first, then implement with parallel sub-agents where possible."

Three Development Workflows

Workflow 1: General Workflow (Single Features)

```
Research → Plan → Implement → Test
```

Research (Optional but valuable for complex features):

- Ask Claude for a research report
- Use external research from ChatGPT Deep Research
- Helpful when making tech stack decisions or using new APIs

Plan (Most important step):

When in plan mode, Claude will:

- Think through the task
- Break it down into steps
- Ask clarifying questions before building

Implement:

- Use the feature dev command: `/feature-dev`
- Claude uses all your configured plugins, MCPs, and commands

Test:

- Run automated tests
- Use browser MCPs for visual testing
- Manual verification

Workflow 2: Multi-Agent Development (Parallel Work)

Run multiple Claude Code instances simultaneously, each on different features.

How it works:

1. **Set up git worktrees**: Multiple working copies of your repo in different directories, each on a different branch
2. **Run separate Claude instances**: Each works in its own worktree with isolated files
3. **Shared git history**: All instances share the same repository
4. **Merge when done**: Combine worktrees into main or a feature branch

Example:

```
# Create worktrees
git worktree add ../project-feature-a feature-a
git worktree add ../project-feature-b feature-b
git worktree add ../project-feature-c feature-c

# Run Claude Code in each directory
cd ../project-feature-a && claude
cd ../project-feature-b && claude
cd ../project-feature-c && claude
```

When to use: Building 2-3 independent features simultaneously.

Four Tips for Productive Building

Tip 1: Use the Best Models

Model	Use For
Opus 4.5	Planning, complex tasks, important decisions
Sonnet	Implementation workhorse, everyday coding
Haiku	Simple tasks, quick bug fixes

Why pay more for better models?

> "The time saved by using a better model that makes fewer mistakes is more valuable than the money saved from using a cheaper model that makes more mistakes."

Tip 2: Periodically Update CLAUDE.md

Don't set it and forget it:

- Update after adding new features
- Update when reaching milestones
- Create a custom command to automate updates

Pro tip: Create a git workflow command that updates CLAUDE.md then commits.

Tip 3: Practice Regression Prevention

When Claude makes a mistake:

1. Don't just fix it and move on
2. Type `#` to add an instruction to CLAUDE.md
3. Claude automatically incorporates it

Example:

```
# Always use the existing Button component from /components/ui/button,
never create new button styles
```

This prevents the same mistake from happening again.

Checklist: Before You Start Coding

- [ ] Answered: What is my goal?
- [ ] Answered: What are my milestones?
- [ ] Created project spec with product requirements
- [ ] Created project spec with technical requirements
- [ ] Set up CLAUDE.md file
- [ ] Set up automated documentation files
- [ ] Installed relevant plugins
- [ ] Connected MCPs for my tech stack
- [ ] Created useful slash commands/sub-agents
- [ ] Configured permissions (optional)
- [ ] Set up hooks (optional)

Remember: The clearer you are, the better Claude understands what you're trying to achieve and builds accordingly.